FedLearn — Cursor AI Build Context

Version: 1.0 | **Date:** May 2026 | **Status:** Ready for Development

> **How to use this file:** Open this file in Cursor and pin it as context (`@FedLearn_Cursor_Context.md`) in every chat session. Reference specific sections with `@section-name` . This document is the single source of truth for architecture, APIs, file structure, and implementation rules.

---

0. Project Identity

Field	Value
Package name	`@fedlearn/core`
Language	Rust (primary) + TypeScript (API layer)
Runtime targets	Native CPU, CUDA (A100), WebAssembly (browser)
Publish target	npm public registry
Framework base	`mni-ml/framework` (autograd tape, existing Rust/CUDA/WebGPU runtime)
Build tool	`cargo` (Rust) + `tsup` (TS bundler)
Node bridge	`napi-rs` (Rust → Node.js native addon)
Test runners	`cargo test` (Rust unit), `jest` (TS integration), custom cross-backend determinism CI
Infra	Modal A100 (training/benchmarks), Cloudflare Workers (aggregation server edge), npm

---

1. Monorepo Structure

```
fedlearn/
├── Cargo.toml                          # Workspace root
├── package.json                        # npm workspace root
├── tsconfig.json
│
├── crates/
│   ├── fedlearn-core/                  # Main Rust crate (compiled to native + WA
│   │   ├── Cargo.toml
│   │   └── src/
│   │       ├── lib.rs
│   │       ├── adapter/                # SECTOR 02 — LoRA adapter
│   │       │   ├── mod.rs
│   │       │   ├── lora.rs             # A/B matrix, inject, merge
│   │       │   ├── dar.rs              # Dynamic Adapter Rank scheduler
│   │       │   └── cold_start.rs       # SVD initialisation for new users
│   │       ├── continual/              # SECTOR 03 — Continual learning
│   │       │   ├── mod.rs
│   │       │   ├── ewc.rs              # Elastic Weight Consolidation (proxy-FIM)
│   │       │   └── gem.rs              # Gradient Episodic Memory
│   │       ├── privacy/                # SECTOR 04 — DP + accounting
│   │       │   ├── mod.rs
│   │       │   ├── dp_sgd.rs           # Fused clip+noise kernel
│   │       │   ├── renyi.rs            # Renyi DP accountant
│   │       │   └── budget.rs           # On-device budget persistence (HMAC)
│   │       ├── session/                # SECTOR 05 — Session isolation
│   │       │   ├── mod.rs
│   │       │   ├── session.rs          # Session struct + lifetime guards
│   │       │   └── ssc.rs              # Semantic Session Compression encoder
│   │       ├── crypto/                 # SECTOR 06 — Secure aggregation
│   │       │   ├── mod.rs
│   │       │   ├── secagg.rs           # X25519 + ChaCha20 + Shamir
│   │       │   └── sketch.rs           # CountSketch gradient filter
│   │       ├── federation/             # SECTOR 07/09 — Client-side federation
│   │       │   ├── mod.rs
│   │       │   ├── client.rs           # FederationClient: round participation
│   │       │   ├── scaffold.rs         # SCAFFOLD optimizer (control variates)
│   │       │   └── distillation.rs     # Federated Distillation path
│   │       └── determinism.rs          # SECTOR 01 — IEEE-754 shim (entry point)
│   │
│   ├── fedlearn-cuda/                  # CUDA kernel crate (feature-gated)
│   │   ├── Cargo.toml
│   │   └── src/
│   │       ├── dp_sgd.cu               # Fused per-sample clip + Gaussian noise
│   │       ├── ewc.cu                  # EWC Fisher diagonal kernel
```

```
│   │       └── lora_merge.cu          # W' = W + A*B fused kernel
│   │
│   ├── fedlearn-wasm/                 # WebAssembly build target
│   │   ├── Cargo.toml
│   │   └── src/
│   │       ├── lib.rs                 # wasm-bindgen exports
│   │       └── browser_session.rs     # Browser-specific session (IndexedDB adap
│   │
│   └── fedlearn-server/               # SECTOR 07 — Aggregation server
│       ├── Cargo.toml
│       └── src/
│           ├── main.rs                # Axum entry point
│           ├── round.rs               # Round lifecycle manager
│           ├── aggregation.rs         # FedAvg / SCAFFOLD / TWA aggregation
│           ├── trust.rs               # Trust-Weighted Aggregation registry
│           └── byzantine.rs           # CountSketch Flame filter
│
├── packages/
│   └── fedlearn-js/                   # TypeScript npm package
│       ├── package.json
│       ├── tsconfig.json
│       ├── src/
│       │   ├── index.ts               # Public API exports
│       │   ├── adapter.ts             # LocalAdapter class
│       │   ├── session.ts             # Session class + types
│       │   ├── privacy.ts             # PrivacyConfig type + dashboard
│       │   ├── federation.ts          # FederationClient wrapper
│       │   └── browser/
│       │       ├── wasm_loader.ts     # WASM module loader
│       │       ├── webgpu_inference.ts # WebGPU forward pass (WGSL shaders)
│       │       └── dashboard.ts       # Epsilon budget UI component
│       └── wgsl/
│           ├── attention.wgsl         # Multi-head attention forward pass
│           └── lora_forward.wgsl      # LoRA adapter forward pass
│
├── benches/                           # Criterion benchmarks
│   ├── dp_sgd_bench.rs
│   ├── ewc_bench.rs
│   └── lora_merge_bench.rs
│
├── tests/
│   ├── determinism/                   # Cross-backend determinism test suite
│   │   ├── reference_vectors.json     # Published TF Privacy test vectors
│   │   └── cross_backend.rs
│   ├── integration/
│   │   ├── federated_round.rs         # 100-device simulated round
│   │   └── adversarial.rs             # Byzantine + dropout simulation
```

```
│   └── privacy/
│       └── accountant_validation.rs   # Renyi DP vs TF Privacy reference
│
├── proxy_corpus/
│   └── public_domain_500.jsonl        # 500-sample public proxy corpus for FIM
│
├── scripts/
│   ├── benchmark_modal.py             # Run full benchmark suite on Modal A100
│   └── simulate_federation.py         # Simulate N-device federated rounds local
│
└── docs/
    ├── ARCHITECTURE.md
    ├── PRIVACY_GUARANTEES.md
    ├── API_REFERENCE.md
    └── BROWSER_GUIDE.md
```

---

2. Core Data Types (Rust)

Define these types first. All other code depends on them.

```rust
// crates/fedlearn-core/src/adapter/lora.rs

/// LoRA adapter: W' = W + A*B  where A: [d, r], B: [r, k]
/// r is dynamic — managed by DarScheduler
pub struct LoraAdapter {
    pub rank: usize,                        // current rank r in [4, 16]
    pub layers: Vec<LoraLayer>,             // one per attention layer
    pub user_id: UserId,
    pub session_count: u64,
    pub budget_remaining: f64,             // epsilon remaining
}

pub struct LoraLayer {
    pub layer_idx: usize,
    pub a: Tensor,                          // shape [d, r]
    pub b: Tensor,                          // shape [r, k]
}

/// Delta produced by session.close() — NOT raw gradients
pub struct LoraDelta {
    pub layers: Vec<LayerDelta>,
    pub session_id: SessionId,
    pub privacy_cost: f64,                 // epsilon consumed this session
}
```

```
pub struct LayerDelta {
    pub layer_idx: usize,
    pub da: Tensor,
    pub db: Tensor,
}


// crates/fedlearn-core/src/session/session.rs

/// Session lifetime is enforced by the Rust type system.
/// 'session lifetime prevents Session from outliving the adapter.
/// Gradients live ONLY in volatile memory — no write path to disk.
pub struct Session<'adapter> {
    adapter: &'adapter mut LoraAdapter,
    ssc_encoder: SscEncoder,               // Semantic Session Compression
    session_id: SessionId,
    interaction_count: usize,
    // PhantomData ensures gradients can't escape the session lifetime
    _volatile: PhantomData<*mut ()>,       // NOT Send + NOT Sync
}

impl<'adapter> Drop for Session<'adapter> {
    fn drop(&mut self) {
        // Guaranteed zeroing of SSC state on session end
        self.ssc_encoder.zero_state();
    }
}


// crates/fedlearn-core/src/privacy/mod.rs

#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct PrivacyConfig {
    pub epsilon: f64,                      // privacy budget per contribution
    pub delta: f64,                        // failure probability (e.g. 1e-5)
    pub clip_norm: f64,                    // L2 gradient clipping threshold
    pub noise_multiplier: f64,             // Gaussian noise sigma
    pub mode: FederationMode,
    pub tier: PrivacyTier,
}

#[derive(Clone, Debug, Serialize, Deserialize)]
pub enum FederationMode {
    Gradient,                              // Share DP-noised gradient deltas
    Distillation,                          // Share soft labels over proxy corpus
}
```

```
#[derive(Clone, Debug, Serialize, Deserialize)]
pub enum PrivacyTier {
    Dp,                                     // DP + SecAgg (default)
    He,                                     // + Homomorphic Encryption (enterpris
}

#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct BudgetState {
    pub total_epsilon: f64,
    pub consumed_epsilon: f64,
    pub round_count: u64,
    pub hmac: [u8; 32],                     // tamper-evident
}
```

---

3. TypeScript Public API

This is the API surface that developers install and call. Keep it minimal and typed.

```
// packages/fedlearn-js/src/index.ts

export { LocalAdapter } from './adapter';
export { Session } from './session';
export type {
  PrivacyConfig, FederationMode, PrivacyTier,
  LoraDelta, BudgetState, InteractionStream,
  AdapterOptions, SessionOptions, ContributeOptions,
} from './privacy';
export { FederationClient } from './federation';
export { EpsilonDashboard } from './browser/dashboard';


// packages/fedlearn-js/src/adapter.ts

export class LocalAdapter {
  /**
   * Load an existing adapter or create a new one for userId.
   * New users get SVD cold-start initialisation from global model weights.
   */
  static async load(userId: string, opts?: AdapterOptions): Promise<LocalAdapter>

  /**
   * Begin a session. All gradients live in volatile memory only.
   * Returns a Session object — call close() when done.
```

```
   */
  beginSession(opts?: SessionOptions): Session;

  /**
   * Apply a LoraDelta to the adapter with EWC forget-resistance.
   * forgetResistance in [0, 1] maps to EWC lambda.
   */
  async apply(delta: LoraDelta, opts?: { forgetResistance?: number }): Promise<vo

  /**
   * Contribute to the global federated model.
   * Applies DP-SGD clipping + noise, then transmits via SecAgg.
   * Deducts epsilon from the local budget.
   */
  async contributeToGlobal(config: PrivacyConfig): Promise<ContributeResult>;

  /** Current privacy budget state. */
  get budget(): BudgetState;

  /** Current LoRA rank (4–16, managed by DAR scheduler). */
  get rank(): number;

  /** Pause/resume federation contributions. Adapter still receives global update
  set contributionEnabled(value: boolean);
}

export interface AdapterOptions {
  rank?: number | 'auto';                  // default: 'auto' (DAR scheduler)
  serverUrl?: string;                      // aggregation server endpoint
  warmUpInteractions?: number;             // default: 5
}

export interface ContributeResult {
  success: boolean;
  epsilonConsumed: number;
  budgetRemaining: number;
  roundId: string;
}


// packages/fedlearn-js/src/session.ts

export class Session {
  /**
   * Feed interactions into the session.
   * Gradients accumulate via SSC encoder in volatile memory — no disk I/O.
   */
```

```
  async learn(interactions: InteractionStream): Promise<void>;

  /**
   * Close the session. Derives LoraDelta from SSC embedding.
   * Raw gradients are DESTROYED. Returns typed delta for adapter.apply().
   */
  async close(): Promise<LoraDelta>;

  /** Number of interactions learned so far this session. */
  get interactionCount(): number;

  /** Whether this session is in warm-up mode (first N interactions). */
  get isWarmingUp(): boolean;
}

export type InteractionStream =
  | AsyncIterable<{ input: string; output: string; loss?: number }>
  | Array<{ input: string; output: string; loss?: number }>;
```

---

4. Implementation Rules (Read Before Coding)

These are hard rules. Cursor must follow all of them.

4.1 Privacy Rules — NON-NEGOTIABLE

- **Session gradients NEVER touch disk.** The `Session` struct must be `!Send + !Sync`. No `Arc<Session>` allowed.
- **EWC Fisher Information Matrix uses ONLY `proxy_corpus/public_domain_500.jsonl`.** Never use user interaction data for FIM computation. This keeps EWC epsilon cost at zero.
- **DP noise is injected ONCE, in the fused CUDA/WASM kernel.** Never inject noise outside `DpSgdKernel::step()`.
- **Budget tracking is HMAC-protected.** `BudgetState` must be signed with device secret on every write. Reject tampered states at startup.
- **`session.close()` must zero the SSC encoder state** before returning the delta (implemented in `Drop`).

4.2 Determinism Rules

- **ALL floating-point operations in the DP noise path go through `DeterministicFloat`**

trait.

- **DP noise uses HKDF(device_secret, round_number) as seed.** PRNG is `ChaCha20Rng`. Never use `rand::thread_rng()` in the privacy path.
- **CI must run `tests/determinism/cross_backend.rs` on every PR.** A single bitwise mismatch = build fail.
- **Never enable FMA fusion** (`#[no_mangle] #[allow(unused)] static DISABLE_FMA: () = ();` in `determinism.rs`).

4.3 Architecture Rules

- **Never add a write path from `Session` to `IndexedDB` or the filesystem.** Only `LoraAdapter` persists to IndexedDB (adapter weights only, never session state).
- **WebGPU = inference only.** Training (DP-SGD) runs on WASM target. Do not attempt WebGPU backprop.
- **`DarScheduler` owns rank.** Never set `adapter.rank` directly from outside the `DarScheduler`.
- **Server never sees plaintext gradients.** SecAgg unmasks only the aggregated sum. Individual updates are always masked when they touch the wire.
- **`FederationMode::Distillation` sends soft labels, not gradients.** The distillation path must NEVER compute or transmit gradient tensors.

4.4 Code Style

- All public Rust types: `#[derive(Debug, Clone, Serialize, Deserialize)]`
- All public Rust errors: use `thiserror::Error`, never `anyhow` in library code
- All async Rust: `tokio` runtime (server) or `wasm-bindgen-futures` (WASM)
- All TypeScript: strict mode, no `any`, export types separately from classes
- All CUDA kernels: one `.cu` file per kernel, one Rust wrapper with `cudarc`
- Benchmarks: `criterion` for Rust, report mean ± std deviation

---

5. Sector Build Order

Build sectors in this exact order. Do not start a sector until all its dependencies pass `cargo`

`test` .

#	Sector	Key Files	Gate
01	Core Autograd & Determinism	`determinism.rs` , `Cargo.toml` workspace	`cross_backend.rs` CI green
02	LoRA Adapter Module	`adapter/lora.rs` , `adapter/dar.rs` , `adapter/cold_start.rs`	`lora_merge.rs` unit tests pass
03	Continual Learning (EWC + GEM)	`continual/ewc.rs` , `continual/gem.rs` , `ewc.cu`	EWC proxy-FIM validated, GEM projection correct
04	DP-SGD & Renyi Accountant	`privacy/dp_sgd.rs` , `privacy/renyi.rs` , `privacy/budget.rs` , `dp_sgd.cu`	Matches TF Privacy vectors at 1e-6
05	Session Isolation + SSC	`session/session.rs` , `session/ssc.rs`	RAM profile benchmark, `!Send` enforced
06	Cryptographic SecAgg	`crypto/secagg.rs` , `crypto/sketch.rs`	20% dropout test passes, WASM build green
07	Aggregation Server	`fedlearn-server/src/*.rs`	100-device load test < 2s/round
08	DAR Scheduler + Cold-Start	`adapter/dar.rs` , `adapter/cold_start.rs`	Rank converges to 6–8 in simulation
09	Byzantine Robustness (TWA)	`fedlearn-server/src/trust.rs` , `byzantine.rs`	< 2% accuracy drop with 10% Byzantine
10	WebGPU / WASM Browser Runtime	`fedlearn-wasm/` , `wgsl/*.wgsl` , `browser/wasm_loader.ts`	Cross-browser test green (Chrome/FF/Edge)
11	Privacy Dashboard	`browser/dashboard.ts` , epsilon UI component	Epsilon counter depletes in real time
12	Integration + npm Publish	All, `scripts/benchmark_modal.py`	All benchmarks pass, `npm publish`

6. Algorithm Specs

Cursor must implement these algorithms exactly. Do not substitute alternatives without explicit instruction.

6.1 LoRA Adapter (Sector 02)

```
Forward pass:   h = W*x + (A*B)*x       where A: [d, r], B: [r, k]
Merge:          W' = W + A*B             (for inference without adapter overhead)
Init (new user): A = top-r left singular vectors of W_attention (via SVD)
                 B = zeros               (so A*B = 0 at init, no disruption)
Rank bounds:    r ∈ {4, 6, 8, 10, 12, 14, 16}
```

6.2 Dynamic Adapter Rank — DAR (Sector 08)

```
After each session close:
  P_before = adapter output distribution on probe_set BEFORE session
  P_after  = adapter output distribution on probe_set AFTER session
  kl = KL(P_before || P_after)

  if kl > tau (default 0.15):
    new_rank = min(rank + 2, 16)
    expand A: pad with top-2 left SV of current A, scaled by sigma
    expand B: pad with corresponding right SV, scaled by sigma

  if kl < tau/3 for 3 consecutive sessions:
    new_rank = max(rank - 2, 4)
    contract A: truncated SVD, keep top-new_rank components
    contract B: corresponding truncation
```

6.3 EWC with Proxy-FIM (Sector 03)

```
Loss = L_task + (lambda/2) * sum_i [ F_i * (theta_i - theta_old_i)^2 ]

Fisher diagonal F_i:
  for each sample s in proxy_corpus (public, 500 samples):
    grad = backward(log_prob(s), theta)
    F_i += grad_i^2
  F_i /= len(proxy_corpus)

NOTE: proxy_corpus is PUBLIC. EWC consumes ZERO epsilon from personal budget.
lambda maps to forgetResistance: lambda = forgetResistance * 5.0
```

6.4 DP-SGD Fused Kernel (Sector 04)

```
For each sample i in batch:
  g_i = gradient of loss_i w.r.t. model params
  g_i_clipped = g_i / max(1, ||g_i||_2 / clip_norm)   # per-sample clip

Aggregate: G = (1/n) * sum(g_i_clipped)
Add noise: G_noisy = G + N(0, sigma^2 * clip_norm^2 * I)
           where sigma = noise_multiplier

CUDA: clip and noise in ONE kernel pass (never two passes — 2x perf)
WASM: same logic in Rust, no SIMD dependency (portable across browsers)
```

6.5 Renyi DP Accountant (Sector 04)

```
Track moments alpha ∈ {2, 4, 8, 16, 32, 64}
Per step: moment_alpha += renyi_divergence(Gaussian mechanism, alpha, sigma, clip
Convert: epsilon(delta) = min over alpha [ (moment_alpha - log(delta)) / (alpha -

Validate against TF Privacy reference vectors (tests/privacy/accountant_validatio
Tolerance: 1e-6 absolute error
```

6.6 Semantic Session Compression — SSC (Sector 05)

```
Encoder architecture:
  Input:  sequence of (loss_t) values, t = 1..T  (variable length)
  Layer 1: Linear(1, 64) + ReLU
  Layer 2: Linear(64, 128) + mean-pool over T
  Output: 128-dim session embedding e_session

Projection head (one per adapter layer):
  delta_A, delta_B = Linear(128, r*d + r*k)(e_session).split()
  delta_A = delta_A.reshape(d, r)
  delta_B = delta_B.reshape(r, k)

RAM: O(128) floats, regardless of session length T
vs full gradient: O(384 * 6 * T) floats for 6-layer, 384-dim transformer
```

6.7 Secure Aggregation Protocol (Sector 06)

```
1. KEY AGREEMENT (per round)
```

```
   Each device pair (i,j): shared_secret_ij = X25519(sk_i, pk_j)
   Mask_ij = ChaCha20(HKDF(shared_secret_ij, round_id), len=gradient_bytes)

2. MASKING
   masked_gradient_i = gradient_i XOR sum_j(Mask_ij * sign(i-j))
   (masks cancel in aggregation: sum(masked) = sum(gradients))

3. COUNTSKETCH (before masking, for Byzantine filter)
   sketch_i = CountSketch(gradient_i, output_dim = input_dim / 100)
   Send sketch_i unencrypted alongside masked_gradient_i

4. DROPOUT RECOVERY
   Use Shamir (k=2/3*n, n) threshold sharing of device secrets
   Server reconstructs masks for dropped devices from surviving shares

5. SERVER AGGREGATION
   Filter by CountSketch cosine similarity (Flame algorithm)
   Unmask sum: aggregate = sum(masked_gradient_i) [masks cancel]
   Apply TWA weights: aggregate_weighted = sum(w_i * gradient_i)
```

6.8 Trust-Weighted Aggregation — TWA (Sector 09)

```
Init: trust_score_i = 0.5 for all devices

After each round:
  delta_loss_i = global_loss_BEFORE - global_loss_AFTER_removing_device_i
  trust_score_i = 0.9 * trust_score_i + 0.1 * sign(delta_loss_i)

Exclusion: if trust_score_i < 0.3 → ban device for 3 rounds

Aggregation weight: w_i = softmax(trust_score_i / temperature)  [temperature = 0.
Final update: W_global += lr * sum(w_i * gradient_i)
```

6.9 SCAFFOLD Optimizer (Sector 07)

```
Each device maintains control variate c_i (same shape as model params)
Server maintains global control variate C

Local update (device i, step t):
  g_t = gradient + (C - c_i)         # correct for client drift
  theta_i -= lr * g_t

After local steps:
  delta_c_i = (theta_i_init - theta_i_final) / (steps * lr) - C
```

```
  c_i_new   = c_i + delta_c_i
  Send: (theta_i_final - theta_i_init, delta_c_i) to server

Server:
  C_new = C + (1/n) * sum(delta_c_i)
  W_global += (1/n) * sum(theta_i_final - theta_i_init)
```

---

7. Cargo.toml — Key Dependencies

```
# crates/fedlearn-core/Cargo.toml
[dependencies]
# ML framework base
mni-ml = { git = "https://github.com/mni-ml/framework", features = ["cuda", "wasm

# Async runtime
tokio = { version = "1", features = ["full"] }

# WASM async
wasm-bindgen-futures = "0.4"
wasm-bindgen = "0.2"

# Crypto
x25519-dalek = "2"
chacha20 = "0.9"
hkdf = "0.12"
sha2 = "0.10"
hmac = "0.12"

# Serialization
serde = { version = "1", features = ["derive"] }
serde_json = "1"
bincode = "1"

# Error handling (library — no anyhow)
thiserror = "1"

# Random (ONLY for DP noise path via HKDF seed)
rand_chacha = "0.3"
rand_core = "0.6"

# Linear algebra (when mni-ml tensor not used)
nalgebra = "0.32"     # for SVD in cold-start initialiser

[features]
```

```
default = ["native"]
native = ["mni-ml/cuda"]
wasm  = ["mni-ml/wasm", "wasm-bindgen", "wasm-bindgen-futures"]
he    = []             # Homomorphic encryption enterprise tier (SEAL-WASM, linke

[dev-dependencies]
criterion = { version = "0.5", features = ["html_reports"] }
approx = "0.5"         # for floating-point test assertions


# crates/fedlearn-server/Cargo.toml
[dependencies]
fedlearn-core = { path = "../fedlearn-core" }
axum = { version = "0.7", features = ["ws"] }
tokio = { version = "1", features = ["full"] }
tower = "0.4"
serde = { version = "1", features = ["derive"] }
serde_json = "1"
thiserror = "1"
tracing = "0.1"
tracing-subscriber = "0.3"
uuid = { version = "1", features = ["v4"] }
```

---

8. CUDA Kernel Specs

dp_sgd.cu — Fused Per-Sample Clip + Noise

```
// One thread per parameter. One pass: clip all samples, sum, add noise.
// Input:  gradients[batch_size][param_dim]  (per-sample gradients)
// Output: noisy_grad[param_dim]
__global__ void dp_sgd_fused(
    const float* __restrict__ grads,      // [batch, dim]
    float* __restrict__ out,              // [dim]
    const float clip_norm,
    const float noise_sigma,
    const float* __restrict__ noise,      // pre-generated Gaussian noise [dim]
    const int batch_size,
    const int param_dim
) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= param_dim) return;

    float sum = 0.0f;
```

```
    for (int b = 0; b < batch_size; b++) {
        // Per-sample L2 norm (computed in shared memory, see full kernel)
        float norm = per_sample_norm[b];   // from shared memory reduction
        float scale = fminf(1.0f, clip_norm / (norm + 1e-8f));
        sum += grads[b * param_dim + idx] * scale;
    }
    out[idx] = (sum / batch_size) + noise_sigma * clip_norm * noise[idx];
}
```

lora_merge.cu — Fused W′ = W + A*B

```
// One kernel: compute A*B and add to W in one pass
// A: [d, r], B: [r, k], W: [d, k]
__global__ void lora_merge(
    const float* __restrict__ A,           // [d, r]
    const float* __restrict__ B,           // [r, k]
    const float* __restrict__ W,           // [d, k]
    float* __restrict__ W_out,             // [d, k]
    const int d, const int r, const int k
) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;   // d dim
    int col = blockIdx.x * blockDim.x + threadIdx.x;   // k dim
    if (row >= d || col >= k) return;

    float ab = 0.0f;
    for (int i = 0; i < r; i++) {
        ab += A[row * r + i] * B[i * k + col];
    }
    W_out[row * k + col] = W[row * k + col] + ab;
}
```

---

9. WGSL Shaders (Browser Inference)

```
// packages/fedlearn-js/wgsl/lora_forward.wgsl
// Compute h = W*x + (A*B)*x for one attention layer

struct LoraParams {
    d: u32,
    r: u32,
    k: u32,
}
```

```
@group(0) @binding(0) var<storage, read>       W: array<f32>;   // [d, k]
@group(0) @binding(1) var<storage, read>       A: array<f32>;   // [d, r]
@group(0) @binding(2) var<storage, read>       B: array<f32>;   // [r, k]
@group(0) @binding(3) var<storage, read>       x: array<f32>;   // [k]
@group(0) @binding(4) var<storage, read_write> h: array<f32>;   // [d]
@group(0) @binding(5) var<uniform>             params: LoraParams;

@compute @workgroup_size(64)
fn main(@builtin(global_invocation_id) gid: vec3<u32>) {
    let row = gid.x;
    if (row >= params.d) { return; }

    var w_x: f32 = 0.0;
    var ab_x: f32 = 0.0;

    for (var j: u32 = 0u; j < params.k; j++) {
        w_x += W[row * params.k + j] * x[j];
    }
    for (var ri: u32 = 0u; ri < params.r; ri++) {
        var b_x: f32 = 0.0;
        for (var j: u32 = 0u; j < params.k; j++) {
            b_x += B[ri * params.k + j] * x[j];
        }
        ab_x += A[row * params.r + ri] * b_x;
    }
    h[row] = w_x + ab_x;
}
```

---

10. Aggregation Server — Axum API

```
// crates/fedlearn-server/src/main.rs

// HTTP endpoints the server exposes
// All endpoints use JSON + WebSocket where noted

// POST /round/register
// Body: { device_id, public_key, trust_score_declared }
// Response: { round_id, peers: [{device_id, public_key}], round_deadline }

// POST /round/submit
// Body: { round_id, device_id, masked_gradient: bytes, sketch: bytes, shamir_sha
// Response: { received: true, position: usize }

// GET /round/result/{round_id}
```

```
// Response: { global_model_delta: bytes, new_round_id, trust_scores_updated }

// WS /round/stream  (WebSocket)
// Server pushes round lifecycle events: started, collecting, aggregating, comple

// POST /distillation/submit
// Body: { round_id, device_id, soft_labels: [[f32]] }  (FD mode only)
// Response: { received: true }
```

---

11. Key Test Cases

Write these tests first (TDD). Sector gates require all relevant tests passing.

```
// Sector 01 — Determinism
#[test]
fn dp_noise_bit_identical_across_cpu_and_wasm() {
    // Generate DP noise on CPU with HKDF seed
    // Generate same DP noise compiled to WASM
    // Assert: every byte identical
}

// Sector 02 — LoRA
#[test]
fn lora_init_does_not_disrupt_base_model() {
    // SVD init: A = top-r SV of W, B = zeros
    // Assert: A*B = zero matrix (no disruption at init)
}

#[test]
fn lora_merge_matches_reference() {
    // Compute W + A*B manually with f64
    // Assert CUDA/CPU kernel matches within 1e-5
}

// Sector 03 — EWC
#[test]
fn ewc_fim_uses_no_user_data() {
    // Run FIM computation, assert zero reads from user interaction store
    // Assert only proxy_corpus is accessed
}

// Sector 04 — DP-SGD
#[test]
fn renyi_accountant_matches_tf_privacy() {
```

```
    // Run accountant for 1000 steps at sigma=1.1, clip=1.0, batch_frac=0.01
    // Assert epsilon at delta=1e-5 within 1e-6 of TF Privacy reference value
}

// Sector 05 — Session
#[test]
fn session_gradients_cannot_be_accessed_after_close() {
    // This should be a COMPILE ERROR, not a runtime error
    // session.close() moves session, making it inaccessible
    // let session = adapter.begin_session();
    // let delta = session.close().await?;
    // session.learn(...);  // <-- should not compile
}

#[test]
fn session_drop_zeros_ssc_state() {
    let ptr = {
        let session = adapter.begin_session();
        session.ssc_encoder.state.as_ptr()
    }; // session dropped here
    // Assert memory at ptr is zeroed (unsafe read after drop, test only)
}

// Sector 06 — SecAgg
#[test]
fn secagg_masks_cancel_in_aggregation() {
    // Simulate N=10 devices with known gradients
    // Apply masks, sum masked gradients
    // Assert sum(masked) == sum(original gradients) within 1e-6
}

#[test]
fn secagg_survives_20_percent_dropout() {
    // 10 devices, 2 drop out before submitting Shamir shares
    // Assert aggregation still produces correct sum using recovered shares
}

// Sector 09 — Byzantine
#[test]
fn twa_excludes_poisoning_device() {
    // 10 devices, 1 sends poisoned gradient (scaled 100x)
    // Run 20 rounds
    // Assert trust_score[poisoner] < 0.3 after round 10
    // Assert global model accuracy degradation < 2%
}
```

12. Benchmark Targets

These are the performance gates. Sector 12 cannot ship until all pass.

Benchmark	Target	Measurement
DP-SGD fused kernel (A100)	< 5ms for batch=64, dim=786K	`criterion` bench
DP-SGD WASM path (M2 browser)	< 50ms for batch=8, dim=786K	`performance.now()`
LoRA merge (CUDA, 6 layers)	< 1ms total	`criterion` bench
EWC FIM computation (proxy corpus)	< 200ms	`criterion` bench
SecAgg round (100 devices, Modal)	< 2s end-to-end	`scripts/benchmark_modal.py`
Browser inference (M2 MacBook, Chrome)	> 10 tokens/sec	`webgpu_bench.ts`
Session RAM (length=50, 6-layer, 384-dim)	< 5MB	`heaptrack`
Adapter serialise/deserialise (r=8)	< 10ms	`criterion` bench
Epsilon accounting: 1000 steps	< 1ms	`criterion` bench

---

13. Environment Setup

```
# Clone and set up
git clone https://github.com/your-org/fedlearn
cd fedlearn

# Rust toolchain
rustup install stable
rustup install nightly                    # needed for some WASM intrinsics
rustup target add wasm32-unknown-unknown

# CUDA (for CUDA kernels — skip if CPU-only dev)
```

```
# Requires CUDA 12.x + cudarc crate
export CUDA_PATH=/usr/local/cuda

# Node / npm
node --version                              # requires 20+
npm install -g tsup tsx

# Install all workspace deps
cargo build                                 # builds all Rust crates
cd packages/fedlearn-js && npm install    # TS package deps

# Run full test suite
cargo test --workspace
cd packages/fedlearn-js && npx jest

# Run determinism gate (required before any PR merge)
cargo test --test cross_backend -- --nocapture

# Build WASM target
cargo build --target wasm32-unknown-unknown --features wasm -p fedlearn-wasm
wasm-bindgen target/wasm32-unknown-unknown/release/fedlearn_wasm.wasm \
  --out-dir packages/fedlearn-js/src/wasm --target web

# Start local aggregation server
cargo run -p fedlearn-server

# Run federation simulation (100 virtual devices)
python scripts/simulate_federation.py --devices 100 --rounds 10
```

---

14. Proxy Corpus Format

The file `proxy_corpus/public_domain_500.jsonl` is used exclusively by EWC FIM computation. Format:

```
{"text": "The quick brown fox jumps over the lazy dog.", "source": "public_domain
{"text": "To be or not to be, that is the question.", "source": "shakespeare"}
... (500 lines total, all public domain text)
```

This file must never be replaced with user data. If it is missing, EWC must fail loudly with a compile-time or runtime error pointing to this requirement.

---

15. Privacy Accounting — Session-Level Budget

```
Each session:
  epsilon_session = renyi_accountant.compute(steps, sigma, clip_norm, batch_frac)
  budget.consumed_epsilon += epsilon_session
  budget.hmac = HMAC-SHA256(device_secret, serialize(budget))
  write budget to IndexedDB (budget only, never session state)

At startup:
  budget = read from IndexedDB
  verify HMAC — if invalid, reset budget to zero and log warning

User-visible:
  budget_remaining = config.epsilon_total - budget.consumed_epsilon
  If budget_remaining <= 0: contributeToGlobal() returns error, does not transmit
```

---

16. Cold-Start Initialisation Flow

```
New user detected (no existing adapter in IndexedDB):

1. Fetch global model weights W_attention for each layer
2. For each layer l:
   a. Compute SVD: W_l = U_l @ S_l @ V_l^T
   b. A_l = U_l[:, :r] * sqrt(S_l[:r])     # top-r left singular vectors, scaled
   c. B_l = zeros(r, k)                      # init to zero → A*B = 0 initially
3. Create LoraAdapter with these A, B matrices
4. Set is_warming_up = true
5. For first 5 sessions: use lr_warmup = 3 * lr_base
6. After 5th session: is_warming_up = false, contribute to federation at normal r

Optional cluster seeding (if server provides centroids):
   - Server exposes GET /cluster/centroids → [{id, adapter_delta}] (8 clusters)
   - Client sends 3 anonymous preference signals → server returns nearest cluster
   - Client applies cluster centroid delta on top of SVD init
```

---

17. What Cursor Should NOT Do

- Do not use `async-std` — use `tokio` everywhere
- Do not use `anyhow` in library crates — use `thiserror`

- Do not use `rand::thread_rng()` in any privacy-related code
- Do not store session state in `IndexedDB`, `localStorage`, or any persistent store
- Do not run EWC Fisher estimation on user interaction data
- Do not attempt WebGPU compute shaders for backpropagation (inference only)
- Do not use `WidthType.PERCENTAGE` in any table rendering
- Do not create a `Session` that is `Send` or `Sync`
- Do not share soft labels in `FederationMode::Gradient` or gradients in `FederationMode::Distillation`
- Do not merge a PR without the determinism gate passing on all three backends

---

18. Glossary

Term	Definition
LoRA	Low-Rank Adaptation — represents weight change as W + A*B where rank r << d,k
EWC	Elastic Weight Consolidation — prevents catastrophic forgetting via Fisher-weighted regularisation
GEM	Gradient Episodic Memory — prevents loss increase on prior tasks via gradient projection
DAR	Dynamic Adapter Rank — scheduler that adjusts LoRA rank based on KL divergence
DP-SGD	Differentially Private SGD — clips per-sample gradients and adds calibrated noise
Renyi DP	Renyi Differential Privacy — tighter composition theorem than (ε,δ)-DP for iterative mechanisms
SecAgg	Secure Aggregation — cryptographic protocol where server learns only the sum of updates
SSC	Semantic Session Compression — fixed-size session encoder replacing raw gradient accumulation

TWA	Trust-Weighted Aggregation — history-based aggregation weight replacing fixed-threshold Byzantine filters
FD	Federated Distillation — federation via soft labels over proxy data instead of gradient deltas
FIM	Fisher Information Matrix — diagonal approximation used by EWC to estimate parameter importance
CountSketch	Dimensionality-reduction hash structure used for Byzantine anomaly detection before unmasking
SCAFFOLD	Federated optimizer using control variates to correct non-IID client drift
FedProx	Federated optimizer adding proximal term μ
HE	Homomorphic Encryption — allows computation on ciphertexts (enterprise privacy tier)
HKDF	HMAC-based Key Derivation Function — used for deterministic DP noise seeding
proxy corpus	Fixed 500-sample public-domain text corpus used exclusively for EWC FIM computation
warm-up	First 5 sessions for a new user: elevated lr, no federation contribution
DXA	Document units (1440 DXA = 1 inch) — used in docx layout calculations
napi-rs	Framework for building Rust-to-Node.js native addons

---

19. Error Types — Complete Hierarchy

Every error in FedLearn must map to one of these types. Never use `unwrap()` in library code. Never use `anyhow` in any crate except `fedlearn-server/src/main.rs` (entry point only).

```
// crates/fedlearn-core/src/lib.rs — top-level error enum

use thiserror::Error;

#[derive(Debug, Error)]
pub enum FedLearnError {
    // ── Adapter errors ──────────────────────────────────────────
```

```
#[error("Adapter not found for user {user_id}")]
AdapterNotFound { user_id: String },

#[error("Adapter rank {rank} out of valid range [4, 16]")]
InvalidRank { rank: usize },

#[error("Rank expansion failed: SVD did not converge for layer {layer_idx}")]
SvdDidNotConverge { layer_idx: usize },

#[error("Cold-start initialisation failed: global model weights unavailable")
ColdStartFailed,

// ── Session errors ──────────────────────────────────────────
#[error("Session already closed — create a new session with adapter.begin_ses
SessionAlreadyClosed,

#[error("Session learn() called with empty interaction stream")]
EmptyInteractionStream,

#[error("SSC encoder state corrupted — session must be abandoned")]
SscStateCorrupted,

// ── Privacy / DP errors ─────────────────────────────────────
#[error("Privacy budget exhausted (consumed {consumed:.4}ε of {total:.4}ε) —
BudgetExhausted { consumed: f64, total: f64 },

#[error("Budget state tampered — HMAC verification failed. Resetting to zero.
BudgetTampered,

#[error("DP-SGD: clip_norm must be > 0, got {clip_norm}")]
InvalidClipNorm { clip_norm: f64 },

#[error("DP-SGD: noise_multiplier must be >= 0, got {noise_multiplier}")]
InvalidNoiseMultiplier { noise_multiplier: f64 },

#[error("Renyi accountant: alpha order {alpha} not tracked (valid: 2,4,8,16,3
InvalidRenyiOrder { alpha: usize },

// ── Proxy corpus errors ─────────────────────────────────────
#[error("Proxy corpus missing at '{path}' — EWC requires public_domain_500.js
ProxyCorpusMissing { path: String },

#[error("Proxy corpus has {found} samples, minimum required is 100")]
ProxyCorpusTooSmall { found: usize },

// ── Crypto / SecAgg errors ──────────────────────────────────
#[error("SecAgg: key agreement failed for peer {peer_id}")]
```

```
    KeyAgreementFailed { peer_id: String },

    #[error("SecAgg: Shamir recovery failed — only {have} shares available, need
    ShamirRecoveryFailed { have: usize, need: usize },

    #[error("SecAgg: masked gradient size mismatch (expected {expected} bytes, go
    MaskedGradientSizeMismatch { expected: usize, got: usize },

    #[error("CountSketch: input dimension {dim} too small for compression ratio 1
    SketchDimensionTooSmall { dim: usize },

    // ── Federation errors ──────────────────────────────────────────
    #[error("Federation round {round_id} expired — deadline was {deadline}")]
    RoundExpired { round_id: String, deadline: String },

    #[error("Server returned unexpected status {status} for round {round_id}")]
    ServerError { status: u16, round_id: String },

    #[error("Federation mode mismatch: server expects {server_mode}, client confi
    FederationModeMismatch { server_mode: String, client_mode: String },

    #[error("Contribution paused by user — set adapter.contribution_enabled = tru
    ContributionPaused,

    // ── Trust errors ───────────────────────────────────────────────
    #[error("Device {device_id} banned from federation for {rounds_remaining} mor
    DeviceBanned { device_id: String, rounds_remaining: u32, score: f64 },

    // ── WASM / Browser errors ──────────────────────────────────────
    #[error("WebGPU not available — falling back to WASM training path")]
    WebGpuUnavailable,

    #[error("IndexedDB unavailable — adapter cannot be persisted in this browser
    IndexedDbUnavailable,

    #[error("WASM module not initialised — call await fedlearn.init() before use"
    WasmNotInitialised,

    // ── IO / Serialisation ─────────────────────────────────────────
    #[error("Serialisation failed: {source}")]
    Serialisation { #[from] source: bincode::Error },

    #[error("IO error: {source}")]
    Io { #[from] source: std::io::Error },
}

// TypeScript surface — these map 1:1 to the Rust errors via napi-rs
```

```
// packages/fedlearn-js/src/errors.ts
export class FedLearnError extends Error {
  constructor(
    message: string,
    public readonly code: FedLearnErrorCode,
    public readonly context?: Record<string, unknown>,
  ) { super(message); this.name = 'FedLearnError'; }
}

export enum FedLearnErrorCode {
  AdapterNotFound       = 'ADAPTER_NOT_FOUND',
  BudgetExhausted       = 'BUDGET_EXHAUSTED',
  BudgetTampered        = 'BUDGET_TAMPERED',
  SessionAlreadyClosed  = 'SESSION_ALREADY_CLOSED',
  ContributionPaused    = 'CONTRIBUTION_PAUSED',
  RoundExpired          = 'ROUND_EXPIRED',
  DeviceBanned          = 'DEVICE_BANNED',
  WebGpuUnavailable     = 'WEBGPU_UNAVAILABLE',
  ProxyCorpusMissing    = 'PROXY_CORPUS_MISSING',
  SvdDidNotConverge     = 'SVD_DID_NOT_CONVERGE',
  ServerError           = 'SERVER_ERROR',
}
```

---

20. CI / CD Pipeline

Every PR must pass all stages. Do not merge if any stage is red.

```
# .github/workflows/ci.yml

name: FedLearn CI

on: [push, pull_request]

jobs:
  # ── Stage 1: Rust unit tests (all platforms) ──────────────────────────
  rust-unit:
    strategy:
      matrix:
        os: [ubuntu-latest, macos-latest, windows-latest]
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@v4
      - uses: dtolnay/rust-toolchain@stable
      - run: cargo test --workspace --exclude fedlearn-cuda
```

```
      # fedlearn-cuda excluded on CPU runners

# ── Stage 2: WASM build ──────────────────────────────────────────
wasm-build:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: dtolnay/rust-toolchain@stable
      with: { targets: wasm32-unknown-unknown }
    - run: cargo build --target wasm32-unknown-unknown --features wasm -p fedle
    - run: |
        npm install -g wasm-bindgen-cli
        wasm-bindgen target/wasm32-unknown-unknown/release/fedlearn_wasm.wasm \
          --out-dir packages/fedlearn-js/src/wasm --target web

# ── Stage 3: DETERMINISM GATE — MUST PASS ────────────────────────
determinism:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: dtolnay/rust-toolchain@stable
      with: { targets: wasm32-unknown-unknown }
    - name: Cross-backend determinism test (CPU vs WASM)
      run: cargo test --test cross_backend -- --nocapture
      # A single bitwise mismatch = CI failure = block merge

# ── Stage 4: TypeScript tests ────────────────────────────────────
typescript:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: { node-version: '20' }
    - run: cd packages/fedlearn-js && npm ci && npx jest --coverage

# ── Stage 5: Privacy accountant validation ───────────────────────
privacy-validation:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: dtolnay/rust-toolchain@stable
    - name: Validate Renyi accountant vs TF Privacy reference
      run: cargo test --test accountant_validation -- --nocapture
      # Fails if any value deviates > 1e-6 from reference

# ── Stage 6: Benchmarks (on PR to main only) ─────────────────────
benchmarks:
```

```
    if: github.base_ref == 'main'
    runs-on: ubuntu-latest   # Modal A100 for CUDA benches — triggered separately
    steps:
      - uses: actions/checkout@v4
      - uses: dtolnay/rust-toolchain@stable
      - run: cargo bench --bench dp_sgd_bench
      - run: cargo bench --bench ewc_bench
      - run: cargo bench --bench lora_merge_bench

  # ── Stage 7: Security audit ──────────────────────────────────────
  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: cargo install cargo-audit
      - run: cargo audit
        # Fails on any RUSTSEC advisory in crypto dependencies
```

---

21. Federation Round Lifecycle — Complete State Machine

This is the authoritative description of what happens during one federated round. Implement `fedlearn-server/src/round.rs` to match this exactly.

```
ROUND STATES:
  WAITING → REGISTERING → COLLECTING → FILTERING → AGGREGATING → DISTRIBUTING → C

──────────────────────────────────────────────────────────────────────────

STATE: WAITING
  - Server is idle between rounds
  - Transition: when round_interval elapses OR min_participants reached
  - Action: Generate new round_id (UUID v4), broadcast ROUND_STARTED event via WS

STATE: REGISTERING  (deadline: 30 seconds)
  - Devices POST /round/register with their public key
  - Server builds peer list, sends each device the full peer list
  - Devices perform X25519 key agreement with all peers (O(n²) key pairs)
  - Devices pre-generate ChaCha20 masks for each peer pair
  - Devices compute CountSketch of their gradient (unencrypted, 100:1 compression
  - Transition: deadline elapsed OR max_participants reached
  - Min participants: 10 (below this, round is cancelled — privacy set too small)

STATE: COLLECTING  (deadline: 60 seconds)
  - Devices POST /round/submit: {masked_gradient, sketch, shamir_share}
  - Server stores all submissions in memory (never disk)
```

```
  - Devices that miss deadline: server triggers Shamir recovery using available s
  - Transition: all registered devices submitted OR deadline elapsed

STATE: FILTERING
  - Server runs CountSketch Flame filter on unencrypted sketches:
      for each pair (i,j): cos_sim = sketch_i · sketch_j / (|sketch_i| * |sketch_
      if device i has mean cos_sim < threshold (0.5): flag as potential Byzantine
  - Trust-Weighted Aggregation lookup: pre-apply w_i = softmax(trust_i / 0.1)
  - Flagged devices: masked gradient excluded, TWA weight set to 0
  - Transition: filtering complete (< 100ms expected)

STATE: AGGREGATING
  - Unmask: sum(masked_gradient_i) = sum(gradient_i)  [masks cancel algebraically
  - Apply TWA weights: W_global += lr * sum(w_i * gradient_i)
  - Update trust scores: trust_i += 0.1 * sign(delta_loss_i) [requires eval pass]
  - For SCAFFOLD mode: update global control variate C
  - For Distillation mode: run server-side distillation step on averaged soft lab
  - Transition: aggregation complete

STATE: DISTRIBUTING
  - Compute global_model_delta = W_global_new - W_global_old
  - Quantise to int8 for transmission (dequantise on device)
  - Broadcast via WS or make available at GET /round/result/{round_id}
  - Include new trust scores for each device (only their own)
  - Transition: all devices acknowledged OR 30s timeout

STATE: COMPLETE
  - Log round metrics: participants, dropouts, Byzantine_filtered, epsilon_consum
  - Clear masked gradients from memory (NEVER persist to disk)
  - Schedule next round (WAITING state)

──────────────────────────────────────────────────────────────────────

FAILURE MODES:
  < 10 participants at COLLECTING deadline → CANCEL round, return to WAITING
  Server crash during AGGREGATING → round lost, devices retry next round
  Byzantine device detected post-round → TWA handles gracefully next round
```

---

22. Browser Integration Guide

How to use `@fedlearn/core` in a browser application. This is the developer-facing integration guide.

22.1 Installation & Initialisation

```
// In your web app's entry point
import { init, LocalAdapter, PrivacyConfig } from '@fedlearn/core/browser';

// Must call init() before any other FedLearn API
// Loads the WASM module and checks WebGPU availability
await init({
  serverUrl: 'wss://your-aggregation-server.com',
  wasmPath: '/fedlearn_wasm_bg.wasm',   // serve from your CDN
});

// WebGPU inference is automatically used if available
// WASM training path is always used (WebGPU backprop not yet stable)
```

22.2 Adapter Lifecycle (Browser)

```
// Adapter persists to IndexedDB automatically
// Session state NEVER touches IndexedDB — volatile memory only

const adapter = await LocalAdapter.load('user-abc-123', {
  rank: 'auto',                          // DAR scheduler manages rank
  serverUrl: 'wss://your-server.com',
  warmUpInteractions: 5,                 // default
});

// Check current budget before contributing
console.log(`Budget: ${adapter.budget.consumed_epsilon.toFixed(3)}ε used`);
console.log(`Remaining: ${adapter.budget.budgetRemaining.toFixed(3)}ε`);

// Begin a session — all gradients stay in RAM
const session = adapter.beginSession();

await session.learn([
  { input: 'What time is the meeting?', output: 'The meeting is at 3pm.' },
  { input: 'Remind me about the report', output: 'Sure, the Q3 report is due Frid
]);

// Close session — derives delta, zeros session memory
const delta = await session.close();

// Apply to personal adapter with forget-resistance
await adapter.apply(delta, { forgetResistance: 0.8 });

// Contribute to global model (deducts from epsilon budget)
const result = await adapter.contributeToGlobal({
  epsilon: 0.5,
```

```
  delta: 1e-5,
  clipNorm: 1.0,
  noiseMultiplier: 1.1,
  mode: 'gradient',
  tier: 'dp',
});

console.log(`Contributed. Budget remaining: ${result.budgetRemaining.toFixed(3)}ε
```

22.3 Privacy Dashboard Component

```
// packages/fedlearn-js/src/browser/dashboard.ts
// Drop-in Web Component — works in any framework

import { EpsilonDashboard } from '@fedlearn/core/browser';

// Vanilla JS
const dashboard = new EpsilonDashboard(adapter);
document.getElementById('privacy-panel').appendChild(dashboard.element);

// React
import { FedLearnDashboard } from '@fedlearn/core/react';
<FedLearnDashboard adapter={adapter} />

// Dashboard renders:
// ┌──────────────────────────────────────────────────┐
// │  Privacy Budget          [Pause contributions]   │
// │  ████████████░░░░░░      3.2ε / 8.0ε used        │
// │  This session: 0.03ε     Lifetime: 47 contributions  │
// │  Federation mode: DP + SecAgg   Tier: Standard   │
// └──────────────────────────────────────────────────┘
```

22.4 Adapter Weight Heat Map

```
// Visualise which attention heads the personal adapter modifies most
import { AdapterHeatMap } from '@fedlearn/core/browser';

const heatmap = new AdapterHeatMap(adapter, {
  canvas: document.getElementById('heatmap-canvas'),
  colorScale: 'viridis',         // low=blue, high=yellow
  updateOnApply: true,           // re-renders after every adapter.apply()
});

// Heat map shows:
```

```
// - 6 rows (attention layers)
// - 12 columns (attention heads)
// - Colour = ||A_l * B_l||_F for that head (Frobenius norm of LoRA delta)
// - Brighter = more personalisation in that head
```

22.5 IndexedDB Schema

```
// Adapter storage schema — ONLY adapter weights, NEVER session state
// Database name: 'fedlearn'
// Object store: 'adapters'

interface AdapterRecord {
  userId: string;                          // primary key
  rank: number;
  layers: {
    layerIdx: number;
    a: Float32Array;                       // serialised A matrix
    b: Float32Array;                       // serialised B matrix
  }[];
  budgetState: {
    totalEpsilon: number;
    consumedEpsilon: number;
    roundCount: number;
    hmac: Uint8Array;
  };
  sessionCount: number;
  lastUpdated: number;                     // Unix timestamp
  version: number;                         // schema version for migrations
}

// Object store: 'global_model_cache'
interface GlobalModelCacheRecord {
  modelId: string;                         // primary key (hash of global weights
  weights: Float32Array;                   // quantised global model weights
  fetchedAt: number;
  expiresAt: number;
}

// NEVER store:
// - Session gradients
// - SSC encoder state
// - Raw interaction text
// - Unnoised gradient deltas
```

23. Distillation Mode — Complete Protocol

When `PrivacyConfig.mode = 'distillation'`, the entire federation path changes. Implement this as a completely separate code path — do not mix gradient and distillation logic.

```
CLIENT SIDE (device):
  1. Load proxy_corpus (same 500-sample public domain corpus as EWC)
  2. Run personal adapter forward pass on each sample:
       for sample s in proxy_corpus:
         logits_s = adapter.forward(s.tokens)
         soft_label_s = softmax(logits_s / temperature=2.0)   // temperature scal
  3. soft_labels = [soft_label_s for s in proxy_corpus]        // shape [500, voc
  4. Quantise to float16 to reduce size
  5. Transmit soft_labels to server via POST /distillation/submit
     NO gradient content. NO masked gradient. NO Shamir shares needed.
     SecAgg is NOT required (soft labels carry negligible gradient information)

SERVER SIDE:
  1. Collect soft_labels_i from all participating devices
  2. Average: avg_labels = mean(soft_labels_i, axis=0)         // shape [500, voc
  3. Run distillation training step on global model:
       for epoch in range(distill_epochs=3):
         for s in proxy_corpus:
           logits_global = global_model.forward(s.tokens)
           loss = KL(softmax(logits_global) || avg_labels[s])  // KL divergence l
           optimizer.step(loss)
  4. Distribute updated global model (same as gradient mode)

PRIVACY NOTE:
  Soft labels cannot be inverted to recover training text (no gradient informatio
  Distillation mode has NO epsilon cost from the gradient mechanism.
  It is NOT free — soft labels do carry some mutual information about training da
  Budget: charge 0.05ε per distillation round as a conservative accounting.

COMMUNICATION COST:
  Gradient mode:   ~150KB per device per round (at r=8, float32)
  Distillation:    ~500 * vocab_size * 2 bytes = ~2MB for 2K vocab
  Use float16 quantisation to bring distillation cost to ~1MB
```

---

24. HE Enterprise Tier — Integration Spec

Only implement this after Sectors 01–09 are complete and stable. It is an additive feature

behind a feature flag.

```
# Enable with: cargo build --features he
[features]
he = ["seal-wasm"]   # SEAL-WASM is a separate C++ library compiled to WASM

[dependencies]
seal-wasm = { path = "../vendor/seal-wasm", optional = true }  # vendored WASM bu


// crates/fedlearn-core/src/privacy/he.rs
// Only compiled when feature "he" is enabled

#[cfg(feature = "he")]
pub struct HeEncryptor {
    context: SealContext,           // BFV scheme, poly_modulus_degree=4096
    public_key: SealPublicKey,      // shared aggregation key from server
    scale: f64,                     // fixed-point scale factor 2^14
}

#[cfg(feature = "he")]
impl HeEncryptor {
    /// Quantise float32 gradient to int16, then encrypt under BFV
    /// Returns ciphertext bytes ready for transmission
    pub fn encrypt_gradient(&self, gradient: &Tensor) -> Result<Vec<u8>, FedLearn
        // 1. Quantise: int_vals = round(gradient * 2^14).clamp(-32768, 32767)
        // 2. Pack int_vals into BFV plaintext (batch encoding)
        // 3. Encrypt with shared public key
        // 4. Serialise ciphertext to bytes
    }
}

// Server side: aggregate ciphertexts directly
// sum_ct = ct_1 + ct_2 + ... + ct_n    (homomorphic addition)
// plaintext_sum = decrypt(sum_ct)       (server decrypts the SUM only)
// gradient_avg = dequantise(plaintext_sum / n)


// TypeScript usage
const result = await adapter.contributeToGlobal({
  epsilon: 0.5,
  delta: 1e-5,
  clipNorm: 1.0,
  noiseMultiplier: 1.1,
  mode: 'gradient',
  tier: 'he',        // activates HE encryption layer
});
```

```
// Note: HE tier still applies DP noise — HE does not replace DP, it augments it
// Total privacy: DP guarantee + cryptographic guarantee (server never sees plain
```

---

25. SSC Encoder — Training Procedure

The SSC encoder is not trained by the end user. It is meta-trained once during Phase 3 and shipped as part of the library. This section defines how to reproduce that training.

```
# scripts/train_ssc_encoder.py
# Run on Modal A100 during Phase 3 build
# Produces: crates/fedlearn-core/src/session/ssc_weights.bin

# Meta-training setup (MAML-style)
# Task: given a simulated user session, predict the correct LoRA delta

# Simulate N=1000 synthetic users with different linguistic patterns
# Each user has a "true adapter" (ground truth LoRA delta after 50 sessions)
# SSC encoder is trained to predict that delta from the session loss sequence alo

# Training loop:
for outer_step in range(10000):
    # Sample a batch of synthetic users
    users = sample_synthetic_users(batch_size=32)

    meta_loss = 0
    for user in users:
        # Inner loop: simulate one session
        session_losses = simulate_session(user, session_length=50)
        predicted_delta = ssc_encoder(session_losses)

        # Compare to ground truth delta via cosine similarity loss
        true_delta = user.ground_truth_delta
        inner_loss = 1 - cosine_sim(predicted_delta, true_delta)
        meta_loss += inner_loss

    # Outer loop: update SSC encoder weights
    meta_optimizer.zero_grad()
    meta_loss.backward()
    meta_optimizer.step()

# After training:
# Export SSC encoder weights to Rust-compatible binary format
# Embed in fedlearn-core as a static byte array (ssc_weights.bin)
# Size: ~200KB (50K params * 4 bytes)
```

26. Cursor Prompt Patterns

When working with Cursor on FedLearn, use these prompt patterns for best results.

Starting a new sector

```
@FedLearn_Cursor_Context.md

I'm starting Sector [N] — [Sector Name].

Prerequisites completed: [list sectors]
Target file: [file path]
Relevant spec sections: [Section X, Section Y]

Build [specific component] following the rules in Section 4.
Use the algorithm from Section [6.X].
Write tests from Section 11 for this component first (TDD).
```

Debugging a test failure

```
@FedLearn_Cursor_Context.md

Test failing: [test name] in [file]
Error: [error message]

Check Section [4/6/11] for the invariant this test is asserting.
Do NOT change the test — the test is the spec. Fix the implementation.
```

Adding a new feature

```
@FedLearn_Cursor_Context.md

I need to implement [feature] on top of [existing component].

Hard constraints from Section 4:
- [relevant rule 1]
- [relevant rule 2]

The feature must not add any persistent state to Session.
Show me what files need to change before writing any code.
```

Cross-backend determinism issue

```
@FedLearn_Cursor_Context.md

Determinism CI failing. CPU and WASM produce different DP noise at step [N].

See Section 4.2 for determinism rules.
See Section 6.4 for the DP-SGD spec.
Check: DeterministicFloat trait applied? HKDF seed correct? FMA disabled?
Do not touch the test — find the source of non-determinism.
```

Performance optimisation

```
@FedLearn_Cursor_Context.md

Benchmark [bench name] is not hitting the target in Section 12.
Current: [X ms], Target: [Y ms]

Profile first. Do not optimise blindly.
The fused CUDA kernel spec is in Section 8 — verify we match it.
Do not sacrifice correctness or the determinism guarantee for speed.
```

---

27. File-by-File Implementation Checklist

Use this as your build tracker. Check off each file as it passes `cargo test` or `jest`.

Sector 01 — Core Autograd & Determinism

```
[ ] crates/fedlearn-core/src/determinism.rs
      - DeterministicFloat trait
      - FMA disable flag
      - IEEE-754 round-to-nearest-even wrapper
      - HKDF-seeded ChaCha20Rng constructor
[ ] tests/determinism/cross_backend.rs
      - CPU vs WASM bitwise comparison
      - 50 reference noise vectors at known seeds
[ ] tests/determinism/reference_vectors.json
      - Pre-generated on CPU, committed to repo
[ ] .github/workflows/ci.yml  (Stage 3 determinism gate)
```

Sector 02 — LoRA Adapter Module

```
[ ] crates/fedlearn-core/src/adapter/lora.rs
      - LoraAdapter struct
      - LoraLayer struct
      - forward() method
      - merge() method
      - Unit tests: init does not disrupt base model, merge correctness
[ ] crates/fedlearn-core/src/adapter/dar.rs
      - DarScheduler struct
      - after_session() → rank decision
      - expand_rank() with SVD-padded init
      - contract_rank() with truncated SVD
      - probe_set loading from disk
      - Unit tests: rank bounds, KL threshold behaviour
[ ] crates/fedlearn-core/src/adapter/cold_start.rs
      - svd_init() from global model weights
      - warm_up_protocol() — elevated lr for 5 sessions
      - Unit tests: A*B = 0 at init, rank preserved after warm-up
[ ] crates/fedlearn-cuda/src/lora_merge.cu
      - Fused W + A*B CUDA kernel
      - Rust wrapper via cudarc
      - Benchmark: lora_merge_bench.rs
```

Sector 03 — Continual Learning

```
[ ] crates/fedlearn-core/src/continual/ewc.rs
      - FisherDiagonal struct
      - compute_fim() — proxy corpus only, never user data
      - ewc_loss() — Fisher-weighted L2 penalty
      - Unit tests: FIM uses no user data, loss shape correct
[ ] crates/fedlearn-core/src/continual/gem.rs
      - EpisodicMemory struct (fixed-size ring buffer)
      - project_gradient() — QP solver
      - Unit tests: loss on replay buffer does not increase
[ ] crates/fedlearn-cuda/src/ewc.cu
      - EWC loss forward pass CUDA kernel
[ ] proxy_corpus/public_domain_500.jsonl
      - 500 samples, validated public domain
      - Unit test: corpus loads, correct line count
```

Sector 04 — DP-SGD & Renyi Accountant

```
[ ] crates/fedlearn-core/src/privacy/dp_sgd.rs
      - DpSgdKernel struct
      - step() — calls CUDA kernel or WASM fallback
```

```
      - per_sample_clip() — L2 norm per sample
      - Unit tests: noise shape, clipping correctness
[ ] crates/fedlearn-core/src/privacy/renyi.rs
      - RenyiAccountant struct
      - step() — moment update for alpha in {2,4,8,16,32,64}
      - compute_epsilon() — conversion to (ε,δ)-DP
      - Unit tests: matches TF Privacy reference at 1e-6
[ ] crates/fedlearn-core/src/privacy/budget.rs
      - BudgetState struct
      - write_to_indexeddb() / read_from_indexeddb()
      - hmac_sign() / hmac_verify()
      - Unit tests: tamper detection, budget exhaustion error
[ ] crates/fedlearn-cuda/src/dp_sgd.cu
      - Fused clip + noise kernel
      - Benchmark: dp_sgd_bench.rs
[ ] tests/privacy/accountant_validation.rs
      - Compare against reference_vectors.json (TF Privacy values)
```

Sector 05 — Session Isolation + SSC

```
[ ] crates/fedlearn-core/src/session/session.rs
      - Session<'adapter> struct
      - PhantomData<*mut ()> (enforces !Send + !Sync)
      - begin() constructor
      - learn() — feeds interaction stream to SSC
      - close() — derives LoraDelta, zeros SSC state, consumes self
      - Drop impl: zero_state()
      - Unit tests: compile-time !Send, zero on drop, close consumes self
[ ] crates/fedlearn-core/src/session/ssc.rs
      - SscEncoder struct (2-layer MLP, 128-dim)
      - forward() — loss sequence → session embedding
      - project() — session embedding → LoraDelta per layer
      - zero_state()
      - load_weights() — from embedded ssc_weights.bin
      - Unit tests: RAM profile at length={10,50,100}
[ ] crates/fedlearn-core/src/session/ssc_weights.bin
      - Committed binary (200KB, from train_ssc_encoder.py output)
```

Sector 06 — Cryptographic SecAgg

```
[ ] crates/fedlearn-core/src/crypto/secagg.rs
      - SecAggClient struct
      - key_agreement() — X25519 with all peers
      - generate_mask() — ChaCha20 from HKDF(shared_secret, round_id)
```

```
      - mask_gradient() — XOR gradient with sum of peer masks
      - shamir_share() — generate shares of device secret
      - Unit tests: masks cancel in aggregation, Shamir recovery correct
[ ] crates/fedlearn-core/src/crypto/sketch.rs
      - CountSketch struct
      - compress() — gradient → sketch (100:1)
      - cosine_similarity() — sketch-space cosine sim
      - Unit tests: compression ratio, similarity ordering preserved
[ ] crates/fedlearn-wasm/src/lib.rs
      - wasm-bindgen exports for SecAgg
      - WASM build verified (no native deps)
      - Integration test: 20% dropout round still correct
```

Sector 07 — Aggregation Server

```
[ ] crates/fedlearn-server/src/main.rs
      - Axum app with all routes wired
      - WebSocket endpoint for round streaming
[ ] crates/fedlearn-server/src/round.rs
      - RoundState enum (all 7 states from Section 21)
      - Lifecycle manager transitions
      - Participant tracking, deadline enforcement
[ ] crates/fedlearn-server/src/aggregation.rs
      - fedavg() — simple average
      - scaffold_aggregate() — with control variate update
      - distillation_step() — KL loss on averaged soft labels
[ ] crates/fedlearn-server/src/trust.rs
      - TrustRegistry struct
      - update() — EMA trust score update
      - ban_check() — exclusion logic
      - weight() — softmax weights for aggregation
[ ] crates/fedlearn-server/src/byzantine.rs
      - flame_filter() — CountSketch cosine similarity filter
      - Integration test: 100-device load test < 2s/round
```

Sector 08 — DAR + Cold-Start (validation pass)

```
[ ] scripts/simulate_federation.py — 1000-user DAR convergence study
[ ] benches/lora_merge_bench.rs — rank transition timing
[ ] Integration test: rank converges to 6–8 over 50 simulated sessions
[ ] Integration test: cold-start user reaches baseline accuracy by session 10
```

Sector 09 — Byzantine Robustness (validation pass)

```
[ ] tests/integration/adversarial.rs
      - 10% Byzantine devices, 20 rounds
      - Assert: global accuracy degradation < 2%
      - Assert: poisoning device trust_score < 0.3 by round 10
      - Assert: CountSketch epsilon overhead < 0.02 per round
```

Sector 10 — WebGPU / WASM Browser Runtime

```
[ ] packages/fedlearn-js/wgsl/attention.wgsl — multi-head attention forward
[ ] packages/fedlearn-js/wgsl/lora_forward.wgsl — LoRA adapter forward pass
[ ] packages/fedlearn-js/src/browser/wasm_loader.ts
      - init() — load WASM module, check WebGPU availability
      - Fallback: if WebGPU unavailable, emit FedLearnError.WebGpuUnavailable
      - Training: always WASM, never WebGPU
[ ] crates/fedlearn-wasm/src/browser_session.rs
      - IndexedDB read/write for LoraAdapter only
      - Verify: session state never written to IndexedDB (compile-time)
[ ] Cross-browser test: Chrome 128+, Firefox 141+, Edge 128+
      - Adapter outputs bit-identical across browsers
      - Benchmark: > 10 tokens/sec on M2 MacBook (Chrome)
```

Sector 11 — Privacy Dashboard

```
[ ] packages/fedlearn-js/src/browser/dashboard.ts
      - EpsilonDashboard class (Web Component)
      - Epsilon progress bar with live depletion
      - Pause toggle (sets adapter.contributionEnabled = false)
      - Federation mode indicator (DP/HE, gradient/distillation)
      - Session activity timeline (last 10 sessions, layers modified)
[ ] packages/fedlearn-js/src/browser/heatmap.ts
      - AdapterHeatMap class
      - Canvas renderer: 6 rows × 12 columns
      - Frobenius norm colormap (viridis)
      - updateOnApply hook
[ ] packages/fedlearn-js/src/react/index.ts (optional)
      - FedLearnDashboard React component wrapper
```

Sector 12 — Integration, Testing & Publish

```
[ ] tests/integration/federated_round.rs
      - 100-device simulated round, 10 global rounds
      - All 5 privacy-utility benchmark points (ε={0.1,0.5,1,4,8})
[ ] scripts/benchmark_modal.py — full A100 benchmark suite
```

```
[ ] All benchmark targets in Section 12 passing
[ ] All cross-backend determinism tests green
[ ] packages/fedlearn-js/src/index.ts — final public API freeze
[ ] docs/API_REFERENCE.md — TSDoc generated reference
[ ] docs/PRIVACY_GUARANTEES.md — mathematical proof summary
[ ] docs/BROWSER_GUIDE.md — integration guide (from Section 22)
[ ] docs/ARCHITECTURE.md — six-layer architecture diagram + explanation
[ ] npm publish @fedlearn/core (public, MIT license)
[ ] Blog: "Differential Privacy from Zero" (1 of 4)
[ ] Blog: "Federated Learning Math" (2 of 4)
[ ] Blog: "Catastrophic Forgetting & EWC" (3 of 4)
[ ] Blog: "Secure Aggregation Protocols" (4 of 4)
```

---

28. Quick Reference — Numbers to Remember

Value	What It Is
`r ∈ [4, 16]`	LoRA rank bounds (DAR scheduler)
`tau = 0.15`	KL divergence threshold for rank expansion
`tau/3 = 0.05`	KL threshold for rank contraction
`3 sessions`	Consecutive sessions below tau/3 needed to contract rank
`5 interactions`	Warm-up period for new users (elevated lr)
`3x`	Warm-up learning rate multiplier
`500 samples`	Proxy corpus size (EWC FIM + distillation)
`0 ε`	EWC epsilon cost (proxy corpus is public)
`0.05 ε`	Conservative epsilon charge per distillation round
`~0.02 ε`	CountSketch Byzantine filter epsilon overhead per round
`128`	SSC session embedding dimension
`50K params`	SSC encoder parameter count (~200KB)
`10 devices`	Minimum round participants (below = cancel round)
`100:1`	CountSketch compression ratio

`0.5`	New device initial trust score
`0.3`	Trust score ban threshold
`3 rounds`	Duration of trust-score ban
`0.9 / 0.1`	Trust score EMA decay / update weights
`0.1`	TWA softmax temperature
`1e-6`	Renyi accountant validation tolerance vs TF Privacy
`10 tokens/sec`	Browser inference performance target (M2 MacBook)
`< 2s`	SecAgg round time target (100 devices, Modal A100)
`< 2%`	Max global accuracy degradation with 10% Byzantine devices
`300KB`	Max adapter size at rank r=16
`75KB`	Min adapter size at rank r=4
`2^14`	HE quantisation scale factor (float32 → int16)
`4096`	SEAL BFV poly_modulus_degree (HE enterprise tier)
`temperature=2.0`	Distillation soft label temperature scaling
`distill_epochs=3`	Server-side distillation training steps per round

---

29. Dependencies — Do Not Change These

These specific library versions are chosen for WASM compatibility, security audit status, and API stability. Do not upgrade without running the full test suite.

Crate / Package	Pinned Version	Why This Version
`x25519-dalek`	`2.0.1`	WASM-compatible, no C deps, audited
`chacha20`	`0.9.1`	Pure Rust, WASM-compatible, no alloc
`hkdf`	`0.12.4`	Pairs with sha2 0.10.x
`sha2`	`0.10.8`	Stable, WASM-compatible

`hmac`	`0.12.1`	For budget HMAC, WASM-compatible
`rand_chacha`	`0.3.1`	Seeded PRNG for DP noise — deterministic
`nalgebra`	`0.32.6`	SVD for cold-start; stable API
`thiserror`	`1.0.58`	Library error handling
`serde`	`1.0.197`	Serialisation — pinned minor
`bincode`	`1.3.3`	Compact binary serialisation for adapter
`axum`	`0.7.5`	Aggregation server
`tokio`	`1.37.0`	Async runtime — LTS
`docx` (npm)	`9.6.1`	Document generation
`typescript`	`5.4.5`	TS compiler
`jest`	`29.7.0`	TS test runner
`tsup`	`8.0.2`	TS bundler for npm publish

This document is the complete and final Cursor AI build context for FedLearn. Pin with @FedLearn_Cursor_Context.md in every session. Last updated: May 2026 — v1.0 final.